Test Plan

Job application workflow: HireKiwi QA Engineer assessment

Document	Test plan with prioritised scenarios
Date	13 October 2026
Systems under test	1. Live portal https://crew.tinobritty.me/jobs (read-only exploratory testing) 2. Local mock application form in mock-app/ (all tests that submit data)
Tools	Playwright Test 1.56.0, Cucumber 13, Node 22; Chromium, Firefox and WebKit

1. Purpose and scope

The goal is to check a job application portal where candidates browse jobs, read the requirements and apply. The work finds defects, confirms expected behaviour, and automates the checks that matter most. The six areas from the brief are the scope. In this document they are called R1 to R6.

Ref	Area	Where it is tested	Test cases
R1	Job listings: titles, salary, experience, description	Live portal, manual	LST-01 to LST-05, LST-11 to LST-13, LST-16 to LST-19
R2	Navigation: job links, Back, unknown addresses, sign-in to apply	Live portal, manual	LST-07 to LST-10, LST-20, LST-21
R3	Form validation: required fields, email, whitespace, limits	Mock form, automated	VAL-01 to VAL-08
R4	Submission: success, duplicates, repeated requests	Mock form, automated	SUB-01 to SUB-05
R5	Error handling: server error, network failure, slow response, rejected field	Mock form, automated	ERR-01 to ERR-04
R6	Cross-browser and small screens	Mock form (automated) and live portal (manual)	XBR-01, XBR-02, XBR-04

2. Approach and testing rules

- **Live portal:** read-only. I browse, search, filter and open jobs. I do not submit applications, save, bookmark, change data or run load tests, as the brief requires.
- **Mock application:** a small local form and server (mock-app/) built so that submission and error cases can be tested safely. It keeps data in memory and uses the same validation rules in the page and on the server.
- **Test design:** positive, negative and boundary cases. Boundary values are tried just below, on and just above each limit (name 2 to 60 characters, experience 0 to 20, cover note 500 characters).
- **Automation:** every automated test checks what the user sees and also asks the server what it really stored. Each test uses its own job ID, so tests never share data and can run in parallel.
- **Evidence:** every automated test saves a screenshot per browser. Live-portal findings have screenshots where one could be taken.

3. How priority was decided

Priority	Meaning	Examples
High (P1)	A candidate could lose an application, send it twice, be blocked from applying, or see wrong key information such as pay.	Empty form, invalid email, double-click submit, duplicate application, server error, network failure, salary shown without a period

Priority	Meaning	Examples
Medium (P2)	Wrong or confusing behaviour that does not stop an application: limits, odd characters, secondary filters, layout.	Name length, cover note limit, Tamil name, skill and category filters, 375 px screen
Low (P3)	Convenience or polish.	Enter key, clickable empty card area, invalid URL values

Reasoning: the application itself is the whole point of the product, so anything that can lose, duplicate or block it comes first. After that comes information the candidate relies on to decide whether to apply (pay, level, description). Presentation and convenience come last. Within a level, I run the cheapest checks first so that a broken build is found early (smoke tests).

4. Prioritised scenarios

High priority (run first)

ID	Scenario	Why it is high	How
VAL-01	Empty form flags every required field and sends nothing	Stops empty applications	Automated
VAL-02	Invalid email formats are rejected	Applications without a way to contact the candidate are useless	Automated
VAL-04	Whitespace-only names are rejected, real names are trimmed	Blank names pass naive "required" checks	Automated
SUB-01	Valid application succeeds and saves exactly one record	The main happy path	Automated
SUB-02	Double-click or repeated submit sends one request	Accidental duplicate requests	Automated
SUB-03	Same email applying twice is rejected, even in capitals	Duplicate applications	Automated
ERR-01	Server error 500 keeps details and allows a retry	Candidate must not lose a long application	Automated
ERR-02	Network failure stops loading and keeps details	Same risk, common on mobile data	Automated
XBR-01	Whole suite gives the same result on Chromium, Firefox and WebKit	Candidates use different browsers	Automated
LST-01	Salary shows its pay period	Pay is key decision information	Manual
LST-03	Card data matches the detail page	Wrong data misleads candidates	Manual
LST-04	Level and Remote filters return the right counts	Main way to narrow the list	Manual
LST-05	Search finds the right job	Main way to find a job	Manual
LST-07	Job links open the right role and Back keeps filters	Core navigation	Manual
LST-15	Filter button opens the filter options	A visible control must work	Manual
LST-16	Detail page shows description, responsibilities and requirements	The brief says candidates view requirements	Manual
LST-21	Signed-out user is asked to sign in and nothing is submitted	Gate to the apply flow	Manual

Medium and low priority

Priority	Cases
Medium (P2)	Manual: LST-02, LST-06, LST-08, LST-11, LST-12, LST-13, LST-17, LST-18, LST-20, XBR-04 Automated: VAL-03, VAL-05, VAL-06, VAL-07, VAL-08, SUB-04, ERR-03, ERR-04, XBR-02
Low (P3)	Manual: LST-09, LST-10, LST-19 Automated: SUB-05

Full steps, expected and actual results are in the Manual Test Cases and Automated Test Cases documents.

5. Cross-browser and compatibility checks

These are the checks that matter most, because the three engines differ most in exactly these places.

Check	Why it matters	Covered by
Whole suite on Chromium, Firefox and WebKit (Safari engine)	Different engines handle forms, focus and validation differently	XBR-01, all Playwright tests
Validation messages and focus move to the first invalid field	Focus behaviour differs between engines	VAL-01
Submit button is disabled while a request is pending	Double-submit protection must work everywhere	SUB-02
Enter key submits once	Keyboard users	SUB-05
Tamil and other non-English text is stored and shown unchanged	Font and encoding differences	VAL-08
No sideways scrolling and a tappable button at 375 px	Most candidates apply from a phone	XBR-02 (form), XBR-04 (live listing)
Real devices (iPhone Safari, Android Chrome) and Edge	Emulation cannot show touch keyboards or autofill	Not covered; recommended next step

6. Environment, entry and exit criteria

- **Environment:** Node 18 or newer (developed on Node 22), Playwright 1.56.0, Cucumber 13, the three Playwright browsers. Setup steps are in the README.
- **Entry:** the mock server starts and answers /healthz; the live portal is reachable.
- **Exit:** all High priority cases have been run; every automated test passes on all three browsers; every failed manual case has a bug report with steps, expected and actual results, severity and evidence where possible.

7. Risks, assumptions and limits

- The mock form is a stand-in. It shows how the tests work, not how the real backend behaves.
- Live-portal results depend on the data on the portal on the day of testing (job counts, salaries). They may change.
- The live portal was tested while signed in, and the sign-in step itself was not exercised, to avoid creating data.
- Load, performance and security penetration testing are out of scope, as the brief forbids load tests.
- The assessment is limited to about 3 to 4 hours, so depth was chosen by priority: High first, then Medium, then Low.

8. Deliverables

Deliverable	Where
Test plan	docs/0_Test_Plan
Manual test cases (21)	docs/1_Manual_Test_Cases
Automated test cases (19 documented, 39 Playwright tests, 38 Cucumber scenarios)	docs/2_Automated_Test_Cases and tests/
Bug reports (8)	docs/3_Bug_Reports
Setup and results	README.md
Screenshots	evidence/live-portal and evidence/automation